

# ***Form Building and Validation***

By Prasertsak U., Ph.D

# Outlines

- Form Building
- Form Elements
- Form Validation
- Exercise

# Form Building

- Using Native Flutter Widgets
  - The standard way to build a form involves grouping multiple form fields into a single container to manage their state and validation ([more info...](#))
- Using the Flutter **Form Builder Package**
  - Pre-made Fields: It includes specialized widgets like *FormBuilderCheckbox*, *FormBuilderDropdown*, *FormBuilderDateTimePicker*, and *FormBuilderSlider*
  - *Standardized Validation: Use the [form\\_builder\\_validators](#) companion package for common checks (e.g., email, numeric, required) without writing custom logic*

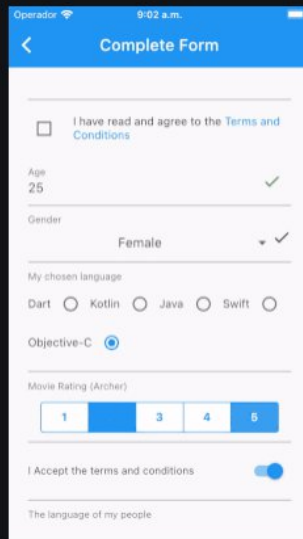
# Form Building (*using a package*)

- Using flutter package : [flutter\\_form\\_builder](#)

## Features

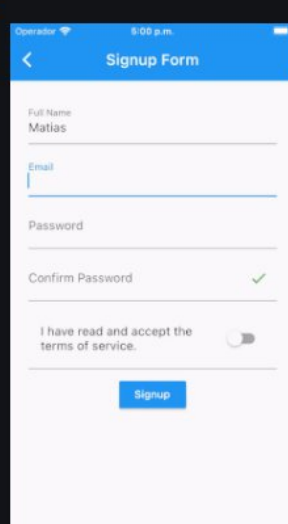
- Create a form with several type of inputs
- Get values from form by easy way
- Apply validators to inputs fields
- React to form fields changes and validations

## Complete Form



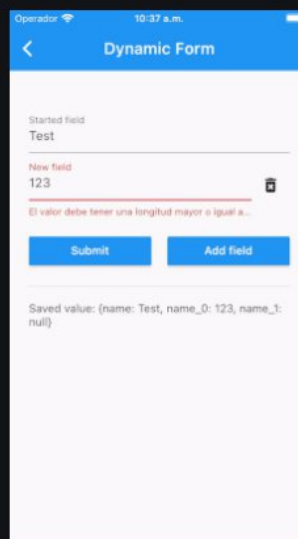
A screenshot of a mobile app interface showing a 'Complete Form' screen. It includes a checkbox for terms and conditions, a text field for age (25), a dropdown for gender (Female), radio buttons for language (Dart, Kotlin, Java, Swift), a radio button for Objective-C, a movie rating slider (Archer) with values 1, 3, 4, 5, a toggle for accepting terms, and a text field for the language of people.

## Sign Up



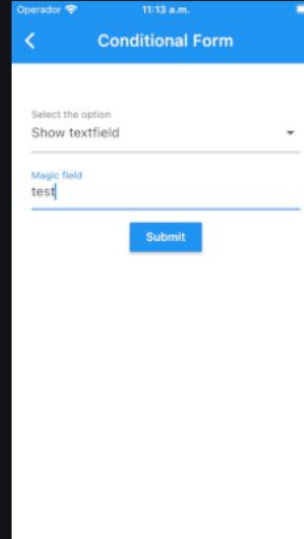
A screenshot of a mobile app interface showing a 'Sign Up' screen. It includes text fields for full name (Matias), email, password, and confirm password, a checkbox for terms and conditions, and a 'Sign up' button.

## Dynamic Fields



A screenshot of a mobile app interface showing a 'Dynamic Form' screen. It includes a 'Started field' (Test), a 'New field' (123) with a validation message 'El valor debe tener una longitud mayor o igual a...', a 'Submit' button, and an 'Add field' button. Below the form, it shows the saved value: (name: Test, name\_0: 123, name\_1: null).

## Conditional Form



A screenshot of a mobile app interface showing a 'Conditional Form' screen. It includes a 'Select the option' dropdown (Show textfield), a 'Magic field' (test), and a 'Submit' button.

## installing

With Flutter:

```
$ flutter pub add flutter_form_builder
```

This will add a line like this to your package's pubspec.yaml

```
dependencies:  
  flutter_form_builder: ^10.3.0+2
```

# การสร้างฟอร์มโดยใช้ flutter\_form\_builder

- เพื่อให้การตรวจสอบความถูกต้องของข้อมูลทั้งหมดภายในฟอร์มหลังจาก submit ทำได้อย่างสะดวก เราจะต้องทำการตรวจสอบไปที่สถานะ ของฟอร์มนั้นๆ โดยการระบุถึงฟอร์มใดๆ นั้น จะต้องอาศัย Key ในการเข้าถึง
- ดังนั้นเราจะทำการสร้าง Key ขึ้นเพื่อใช้ในการกำหนดให้กับฟอร์มผ่าน key properties โค้ดหมายเลข (1) คือการสร้าง key
- โค้ดหมายเลข (2) คือการนำ Key ไปใช้กับ Form ที่กำลังใช้งาน

```
14 · final _formKey = GlobalKey<FormBuilderState>(); ← 1
15 ·
16 · @override
17 · Widget build(BuildContext context) {
18 ·   return Scaffold(
19 ·     appBar: AppBar(
20 ·       centerTitle: true,
21 ·       title: const Text('BMR Calculator'),
22 ·       backgroundColor: Colors.deepPurple,
23 ·       foregroundColor: Colors.white,
24 ·     ), // AppBar
25 ·     body: SingleChildScrollView(
26 ·       child: Container(
27 ·         padding: const EdgeInsets.all(16.0),
28 ·         child: FormBuilder(
29 ·           key: _formKey, ← 2
30 ·           child: Column(
31 ·             children: [
32 > ..... FormBuilderRadioGroup<String>( // FormBuilderRadioGroup ·
53 > ..... SizedBox(height: 16.0),
54 > ..... FormBuilderTextField( // FormBuilderTextField ...
72 > ..... SizedBox(height: 16.0),
73 > ..... FormBuilderTextField( // FormBuilderTextField ...
91 > ..... SizedBox(height: 16),
92 > ..... FormBuilderSlider( // FormBuilderSlider ...
115 > ..... SizedBox(height: 24),
116 > ..... Container( // Container ...
149 ..... ],
150 ..... ), // Column
151 ..... ), // FormBuilder
152 ..... ), // Container
153 ..... ), // SingleChildScrollView
154 ..... ); // Scaffold
155 }
```

# Form Builder : TextField

```
22  FormBuilderTextField(  
23      name: 'name',  
24      decoration: InputDecoration(  
25          labelText: 'Name',  
26          icon: Icon(Icons.person),  
27          iconColor: Colors.deepPurple,  
28      ), // InputDecoration  
29      onChanged: (value) {  
30          person.name = value;  
31      },  
32  ), // FormBuilderTextField  
33  FormBuilderTextField(  
34      name: 'email',  
35      decoration: InputDecoration(  
36          labelText: 'Email',  
37          icon: Icon(Icons.email),  
38          iconColor: Colors.deepPurple,  
39      ), // InputDecoration  
40      onChanged: (value) {  
41          person.email = value;  
42      },  
43  ), // FormBuilderTextField
```

## Form Builder Example

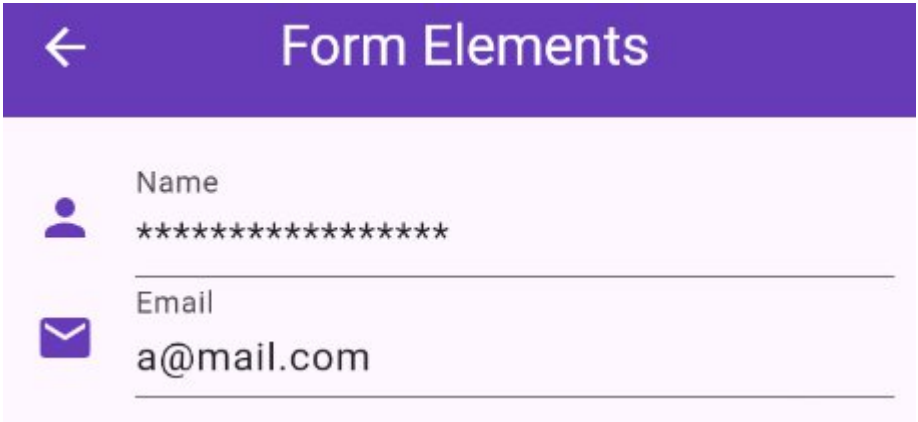
 Name

 Email

# Form Builder : **TextField** – Secured

- เราสามารถปิดการมองเห็นตัวอักษรที่กรอกในกรณีที่ต้องการความปลอดภัย เช่น กรณีกรอกรหัสผ่าน เป็นต้น

```
31 | ..... | FormBuilderTextField(  
32 | ..... |   name: 'name',  
33 | ..... |   obscureText: true,  
34 | ..... |   obscuringCharacter: '*',  
35 | ..... |   decoration: InputDecoration(  
36 | ..... |     labelText: 'Name',  
37 | ..... |     icon: Icon(Icons.person),  
38 | ..... |     iconColor: Colors.deepPurple,  
39 | ..... |   ), // InputDecoration  
40 | ..... |   onChanged: (value) {  
41 | ..... |     person.name = value;  
42 | ..... |   },
```



← Form Elements

Name  
\*\*\*\*\*

Email  
a@mail.com

# Form Builder : RadioGroup

```
44 FormBuilderRadioGroup<String>(  
45     name: 'gender',  
46     decoration: const InputDecoration(labelText: 'Gender:'),  
47     orientation: OptionsOrientation.vertical,  
48     options: ['Male', 'Female', 'Other']  
49     .map(  
50         (yourGender) => FormBuilderFieldOption(  
51             value: yourGender,  
52             child: Text(yourGender),  
53         ), // FormBuilderFieldOption  
54     )  
55     .toList(),  
56     onChanged: (value) {  
57         person.gender = value;  
58     },  
59 ), // FormBuilderRadioGroup
```

orientation: OptionsOrientation.horizontal,



## Form Builder Example

 Name  
\_\_\_\_\_  
 Email  
\_\_\_\_\_  
Gender:  
☒ Male  
☐ Female  
☐ Other  
\_\_\_\_\_

## Form Builder Example

 Name  
\_\_\_\_\_  
 Email  
\_\_\_\_\_  
Gender:  
☐ Male ☐ Female ☐ Other  
\_\_\_\_\_

## Form Builder : Slider

```
61 | ..... | FormBuilderSlider(  
62 | ..... |     name: 'age',  
63 | ..... |     decoration: InputDecoration(labelText: 'Age:'),  
64 | ..... |     min: 0,  
65 | ..... |     max: 100,  
66 | ..... |     divisions: 100,  
67 | ..... |     initialValue: 25,  
68 | ..... |     onChanged: (value) {  
69 | ..... |         person.age = value?.toInt();  
70 | ..... |     },  
71 | ..... | ), // FormBuilderSlider
```



## Form Builder Example

 Name

 Email

Gender:

☐ Male ☐ Female ☐ Other

Age:

070100

# Form Builder : DropDown

```
73 ..... FormBuilderDropdown<String>(  
74 .....   name: 'country',  
75 .....   decoration: InputDecoration(labelText: 'Country:'),  
76 .....   items: ['Thailand', 'China', 'Japan', 'Australia', 'Other']  
77 .....     .map(  
78 .....       (countryName) => DropdownMenuItem(  
79 .....         value: countryName,  
80 .....         child: Text(countryName),  
81 .....       ), // DropdownMenuItem  
82 .....     )  
83 .....     .toList(),  
84 .....     onChanged: (value) {  
85 .....       person.country = value;  
86 .....     },  
87 .....   ), // FormBuilderDropdown
```

**Form Builder Example**

 Name  
\_\_\_\_\_

 Email  
\_\_\_\_\_

Gender:  
☐ Male ☐ Female ☐ Other

Age:  
  
0 25 100

Country:  
Thailand ▼



**Form Builder Example**

 Name  
\_\_\_\_\_

 Email  
\_\_\_\_\_

Gender:  
☐ Male ☐ Female ☐ Other

Age:  
  
0 25 100

Thailand

China

Japan

Australia

Other

# Form Builder : CheckBox

```
89 ..... FormBuilderCheckboxGroup<String>(  
90 .....   name: 'favoriteActivities',  
91 .....   decoration: const InputDecoration(  
92 .....     labelText: 'The Favorite Activities',  
93 .....   ), // InputDecoration  
94 .....   // initialValue: const ['Watching TV', 'Playing Games'],  
95 .....   options: const [  
96 .....     FormBuilderFieldOption(value: 'Listening to Music'),  
97 .....     FormBuilderFieldOption(value: 'Painting'),  
98 .....     FormBuilderFieldOption(value: 'Playing Games'),  
99 .....     FormBuilderFieldOption(value: 'Watching TV'),  
100 .....     FormBuilderFieldOption(value: 'Cooking'),  
101 .....   ],  
102 .....   onChanged: (value) {  
103 .....     person.favoriteActivities = value;  
104 .....   },  
105 ..... ), // FormBuilderCheckboxGroup
```



### Form Builder Example

 Name

 Email

Gender:  
☐ Male ☐ Female ☐ Other

Age:  

025100

Country:  
Thailand 

The Favorite Activities  

☐ Listening to Music ☐ Painting

☒ Playing Games ☒ Watching TV

☒ Cooking

# Form Builder : DatePicker

```
109 ..... FormBuilderDateTimePicker(  
110 .....   name: 'registrationDate',  
111 .....   decoration: InputDecoration(  
112 .....     labelText: 'Registration Date:',  
113 .....     prefixIcon: Icon(Icons.calendar_today),  
114 .....     prefixIconColor: Colors.deepPurple,  
115 .....   ), // InputDecoration  
116 .....   inputType: InputType.date,  
117 .....   format: DateFormat('dd-MM-yyyy'),  
118 .....   onChanged: (value) {  
119 .....     if (value != null) {  
120 .....       person.registrationDate = value.toString();  
121 .....     }  
122 .....   },  
123 ..... ), // FormBuilderDateTimePicker
```

การใช้ DateFormat() จะต้องใช้งาน package ชื่อ **intl** ซึ่งต้องระบุไว้ใน **pubspec.yaml**

```
36 ..... cupertino_icons: ^1.0.8  
37 ..... flutter_form_builder: ^10.3.0+2  
38 ..... form_builder_validators: ^11.2.0  
39 ..... intl: ^0.20.2  
40 .....
```

Form Builder Example

Select date

Mon, Apr 13 

April 2026  

| S  | M  | T  | W  | T  | F  | S  |
|----|----|----|----|----|----|----|
|    |    |    | 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  | 9  | 10 | 11 |
| 12 | 13 | 14 | 15 | 16 | 17 | 18 |
| 19 | 20 | 21 | 22 | 23 | 24 | 25 |
| 26 | 27 | 28 | 29 | 30 |    |    |

Cancel OK

13-04-2026

# Form Builder : TimePicker

```
125 FormBuilderDateTimePicker(  
126   name: 'registrationTime',  
127   decoration: InputDecoration(  
128     labelText: 'Registration Time:',  
129     prefixIcon: Icon(Icons.access_time),  
130     prefixIconColor: Colors.deepPurple,  
131   ), // InputDecoration  
132   inputType: InputType.time,  
133   format: DateFormat('HH:mm'),  
134   onChanged: (value) {  
135     if (value != null) {  
136       person.registrationTime = value.toString();  
137     }  
138   },  
139 ), // FormBuilderDateTimePicker
```



Form Builder Example

Select time

2 : 05

AM  
PM

11 12 1  
10 9 8 7 6 5 4 3

Cancel OK

Registration Time:  
14:05

# การนำค่าที่กรอกบน Form ไปใช้งาน

- ใช้ onChange เมื่อต้องการทำงานแบบเรียลไทม์ ทุกครั้งที่มีการพิมพ์ หรือเปลี่ยนค่า เหมาะสำหรับการทำ Auto-validate หรืออัปเดต UI ทันที
- ส่วน onSave จะทำงานเฉพาะตอนที่สั่ง `_formKey.currentState!.save()` เพื่อนำข้อมูลไปเก็บในตัวแปร เหมาะสำหรับจังหวะที่ผู้ใช้กดปุ่มบันทึก (Submit)

| คุณสมบัติ      | onChanged                                                                                   | onSaved                                                                                     |
|----------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| จังหวะการทำงาน | ทำงานทันทีทุกครั้งที่มีการพิมพ์หรือเปลี่ยนค่าตัวอักษรทีละตัว                                | ทำงานเมื่อเรียกใช้คำสั่ง <code>_formKey.currentState!.save()</code> มักใช้คู่กับปุ่ม Submit |
| จุดประสงค์หลัก | อัปเดต UI ทันที, ค้นหาข้อมูล (Search), หรือเช็คเงื่อนไขเบื้องต้น (เช่น เช็คความยาวรหัสผ่าน) | ดึงข้อมูลจากทุก Field ลงตัวแปร (Model) รวบรวมเพื่อเตรียมส่งไป API หรือฐานข้อมูล             |

# การนำค่าที่กรอกบน Form ไปใช้งาน (cont.)

```
String _name = '';

TextFormField(
  // 1. onChanged ทำงานทุกตัวอักษรที่พิมพ์
  onChanged: (value) {
    print('กำลังพิมพ์: $value');
  },

  // 2. onSaved ทำงานเมื่อฟอร์มถูกส่งให้บันทึก (Save)
  onSaved: (value) {
    _name = value ?? '';
  },
)

// ตัวอย่างการทริกเกอร์ onSaved ผ่านปุ่ม
ElevatedButton(
  onPressed: () {
    if (_formKey.currentState!.validate()) {
      _formKey.currentState!.save(); // เรียก onSaved ของทุก Field
      print('บันทึกชื่อ: $_name');
    }
  },
  child: Text('บันทึกข้อมูล'),
)
```

# Form Validation

- กระบวนการตรวจสอบความถูกต้องของข้อมูลที่ใช้กรอกลงในแบบฟอร์มบนเว็บไซต์หรือแอปพลิเคชัน ก่อนที่จะส่งข้อมูลไปยังเซิร์ฟเวอร์ เพื่อให้แน่ใจว่าข้อมูลถูกต้อง ครบถ้วน และปลอดภัย เช่น การตรวจสอบการมีอยู่ของข้อมูล (Required), รูปแบบอีเมล, หรือความยาวรหัสผ่าน เป็นต้น

Billing address

|                      |                      |
|----------------------|----------------------|
| ชื่อ                 | นามสกุล              |
| <input type="text"/> | <input type="text"/> |
| โปรดระบุชื่อจริง     | โปรดระบุนามสกุล      |

# Form Validation (cont.)

- ทำการหุ้มหรือครอบ Form Elements ทั้งหมดไว้ใน Widget ชื่อว่า **Form** และทำการกำหนด key ของ Form นั้นๆ เพื่อใช้ในการอ้างถึงในการตรวจสอบ state ของ form ว่าทุก input elements ภายใน Form นั้นถูกต้องตามที่ต้องการหรือไม่
- การทำ Validate สามารถแยกทำเป็นราย input element ได้ แต่หาก... ต้องการตรวจสอบในภาพรวมของทุก elements ภายใน Widget **Form** ที่หุ้มหรือครอบไว้จะตรวจสอบได้จาก currentState ของ Form
- การตรวจสอบสามารถกระทำได้หลายลักษณะ เช่น
  1. การเขียนส่วนตรวจสอบไว้ภายในส่วนของ Field นั้น ๆ (เหมาะกับ Form ขนาดเล็กที่ไม่ได้มีการตรวจสอบจำนวนมาก)
  2. การเขียนโปรแกรมตรวจสอบแยกส่วนเป็น Class ย่อยสำหรับเรียกใช้ซ้ำได้
  3. การใช้งาน Library เสริมเพื่อช่วยในการตรวจสอบ

# ตัวอย่างการเขียน Validation ไว้ภายใน input element

```
72 child: TextFormField(  
73   decoration: InputDecoration(  
74     labelText: 'Enter number #1',  
75     border: OutlineInputBorder(),  
# 76     prefixIcon: Icon(Icons.numbers),  
77   ), // InputDecoration  
78   validator: (value) {  
79     if (value == null || value.isEmpty) {  
80       return 'Please enter a number';  
81     }  
82     final numValue = double.tryParse(value);  
83     if (numValue == null) {  
84       return 'Please enter a valid number';  
85     }  
86     return null;  
87   },  
88   onChanged: (value) {  
89     number1 = double.tryParse(value);  
90   },  
91 ), // TextFormField
```

Flutter Demo Home Page

Enter number #1  
# 222

Enter number #2  
# aaa

Please enter a valid number

Add

# ตัวอย่างการสร้าง Class Validator ไว้ใช้เอง

```
import 'package:flutter/material.dart';

class Validators {
  static FormFieldValidator<String> required({errorMessage}) {
    return (String? value) {
      if (value == null || value.isEmpty) {
        return errorMessage;
      }
      return null;
    };
  }

  static FormFieldValidator<String> numberValidator({errorMessage}) {
    return (String? value) {
      final numValue = double.tryParse(value!);
      if (numValue == null) {
        return errorMessage;
      }
      return null;
    };
  }
}
```

ตัวอย่าง code เต็ม [ที่นี่](#)

# ตัวอย่างการนำ Class Validator ไปใช้งาน

```
1 import 'package:flutter/material.dart';
```

```
2 import 'validators.dart';
```

```
3
```

Run | Debug | Profile

```
4 void main() {
```

```
5   | runApp(const MyApp());
```

```
6 }
```

```
child: TextFormField(  
  decoration: InputDecoration(  
    labelText: 'Enter number #2',  
    border: OutlineInputBorder(),  
    prefixIcon: Icon(Icons.numbers),  
  ), // InputDecoration  
  validator: Validators.multiValidator([  
    Validators.required(errorMessage: 'Please enter a number'),  
    Validators.minValue(minValue: 0, errorMessage: 'Number must be greater than or equal to 0'),  
    Validators.numberValidator(errorMessage: 'Please enter a valid number'),  
  ]),  
  onChanged: (value) {  
    number2 = double.tryParse(value);  
  },  
), // TextFormField
```

ตัวอย่างการเรียกใช้แบบหลายเงื่อนไข  
พร้อมกัน

```
child: TextFormField(  
  decoration: InputDecoration(  
    labelText: 'Enter number #1',  
    border: OutlineInputBorder(),  
    prefixIcon: Icon(Icons.numbers),  
  ), // InputDecoration  
  validator: Validators.required(errorMessage: 'Please enter a number'),  
  onChanged: (value) {  
    number1 = double.tryParse(value);  
  },  
), // TextFormField
```

ตัวอย่างการเรียกใช้แบบเงื่อนไขเดียว

# Form Validation แบบใช้ Library เสริม

- การตรวจสอบความถูกต้องของข้อมูลด้วยการใช้งาน package ชื่อ [form\\_builder\\_validators](#)

form\_builder\_validators 11.3.0 

Published 2 months ago •  flutterformbuilderecosystem.com

SDK | FLUTTER | PLATFORM | ANDROID | IOS | LINUX | MACOS | WEB | WINDOWS

 373

[Readme](#) | Changelog | Example | Installing | Versions | Scores

## Form Builder Validators

Form Builder Validators set of validators for any `FormField` widget or widgets that extend the `FormField` class - e.g., `TextFormField`, `DropdownFormField`, *et cetera*. It provides standard ready-made validation rules and a way to compose new validation rules combining multiple rules, including custom ones.

# Ready-made Validation rules

- This package comes with several most common FormFieldValidators such as `required`, `numeric`, `mail`, `URL`, `min`, `max`, `minLength`, `maxLength`, `minWordsCount`, `maxWordsCount`, `IP`, etc., with default `errorText` messages
- อ่านข้อมูลเพิ่มเติมได้ [ที่นี่](#)

# การอ้างอิง Library เพื่อใช้งาน Form Validation

- การใช้งาน Validator ให้ระบุชื่อ package ไว้ภายใน pubspec.yaml และทำการ import เมื่อต้องการใช้

pubspec.yaml



```
36  · cupertino_icons: ^1.0.8
37  · flutter_form_builder: ^10.3.0+2
38  · form_builder_validators: ^11.2.0
39  · intl: ^0.20.2
```



```
1  import 'package:flutter/material.dart';
2  import 'package:flutter_form_builder/flutter_form_builder.dart';
3  import 'package:form_builder_validators/form_builder_validators.dart';
4  import 'package:intl/intl.dart';
5  import '../model/personal.dart';
```

# การใช้งาน Form Validation กับ TextField

```
31 | ..... | FormBuilderTextField(  
32 | ..... |   name: 'name',  
33 | ..... |   decoration: InputDecoration(  
34 | ..... |     labelText: 'Name',  
35 | ..... |     icon: Icon(Icons.person),  
36 | ..... |     iconColor: Colors.deepPurple,  
37 | ..... |   ), // InputDecoration  
38 | ..... |   onChanged: (value) {  
39 | ..... |     person.name = value;  
40 | ..... |   },  
41 | ..... |   autovalidateMode: AutovalidateMode.always,  
42 | ..... |   validator: FormBuilderValidators.compose([  
43 | ..... |     FormBuilderValidators.required(errorText: 'Name is required.'),  
44 | ..... |   ]),  
45 | ..... | ), // FormBuilderTextField
```



เพิ่มส่วนของ validator เข้าไปใน FormBuilderTextField ที่เราต้องการตรวจสอบ โดยเราสามารถตรวจสอบด้วยเงื่อนไขเดียว หรือหลายเงื่อนไขก็ได้ ในตัวอย่าง ใช้รูปแบบการตรวจสอบหลายเงื่อนไข แต่ระบุเงื่อนไขไว้เพียงตัวเดียว คือกำหนดให้ผู้ใช้จะต้องกรอกข้อมูลเท่านั้นห้ามปล่อยเป็นค่าว่าง

ในส่วนของ autovalidateMode กำหนดให้เป็น always นั้น เพื่อให้การตรวจสอบทำทันทีหลังจาก Form นี้ถูกแสดง เพราะหากไม่ใส่ไว้ การตรวจสอบจะต้องทำในช่วงก่อน Submit แต่การตรวจสอบทันทีจะช่วยให้ผู้ใช้กรอกข้อมูลได้ถูกต้องตั้งแต่เริ่ม

# การใช้งาน Form Validation กับ TextField

```
46 | ..... | FormBuilderTextField(  
47 | ..... |   name: 'email',  
48 | ..... |   decoration: InputDecoration(  
49 | ..... |     labelText: 'Email',  
50 | ..... |     icon: Icon(Icons.email),  
51 | ..... |     iconColor: Colors.deepPurple,  
52 | ..... |   ), // InputDecoration  
53 | ..... |   onChanged: (value) {  
54 | ..... |     person.email = value;  
55 | ..... |   },  
56 | ..... |   autovalidateMode: AutovalidateMode.always,  
57 | ..... |   validator: FormBuilderValidators.compose([  
58 | ..... |     FormBuilderValidators.required(errorText: 'Email is required.'),  
59 | ..... |     FormBuilderValidators.email(errorText: 'Please enter a valid email address.'),  
60 | ..... |   ]),  
61 | ..... | ), // FormBuilderTextField
```

← Form Elements

 Name

Name is required.

 Email

Email is required.

ในตัวอย่างทำการตรวจสอบหลายเงื่อนไข ประกอบด้วย 1.จะต้องกรอกข้อมูลป้อนว่างไม่ได้ และ 2.ข้อมูล email จะต้องจัดรูปแบบให้ถูกต้อง ซึ่งในแต่ละเงื่อนไข จะสามารถระบุข้อความแจ้งเตือนได้อิสระต่อกัน

# การใช้งาน Form Validation (cont.)

```
82 | ..... | FormBuilderSlider(  
83 | ..... |   name: 'age',  
84 | ..... |   decoration: InputDecoration(labelText: 'Age:'),  
85 | ..... |   min: 0,  
86 | ..... |   max: 100,  
87 | ..... |   divisions: 100,  
88 | ..... |   initialValue: 25,  
89 | ..... |   onChanged: (value) {  
90 | ..... |     person.age = value?.toInt();  
91 | ..... |   },  
92 | ..... |   autovalidateMode: AutovalidateMode.always,  
93 | ..... |   validator: FormBuilderValidators.compose([  
94 | ..... |     FormBuilderValidators.required(errorText: 'Age is required.'),  
95 | ..... |     FormBuilderValidators.min(1, errorText: 'Age must be at least 1')  
96 | ..... |   ]),  
97 | ..... | ), // FormBuilderSlider
```



ในตัวอย่างทำการตรวจสอบข้อมูลที่เป็นตัวเลข ประกอบด้วย

1. จะต้องกรอกข้อมูลปล่อยว่างไม่ได้ และ
2. ข้อมูลที่กรอกจะต้องมากกว่า 0

Form Elements

Name  
Prasertsak

Email  
abc@mail.com

Gender:  
☒ Male ☐ Female ☐ Other

Age:  
 0 0 100  
Age must be at least 1

Country:  
▼

# ทดสอบการทำงานจากตัวอย่าง

- ให้สร้างโปรเจกต์ชื่อ ***form\_example***
- เพิ่ม package ที่ต้องใช้ลงใน ***pubspec.yaml***

```
36  · cupertino_icons: ^1.0.8
37  · flutter_form_builder: ^10.3.0+2
38  · form_builder_validators: ^11.2.0
39  · intl: ^0.20.2
```

- ให้ download โปรแกรมตัวอย่างจาก [ที่นี่](#)
- แยกไฟล์ ZIP และนำโค้ดจาก ***/lib*** วางทับลงในโปรเจกต์ที่สร้างขึ้น
- ทดสอบการทำงานด้วยการรัน ***main.dart***

# ทดสอบความเข้าใจ

- ให้สร้างโปรเจกต์ใหม่ชื่อ **bodyhealth\_app**
- สร้าง folder ชื่อว่า **models** และ **pages** ภายใน **/lib**
- สร้างไฟล์ชื่อ **bmr\_pages.dart** ไว้ภายใน **pages**
- สร้างไฟล์ชื่อ **body\_profile.dart** ไว้ภายใน **models** โดยมีโครงสร้างประกอบด้วย
  - weight – **double**
  - height - **double**
  - age - **int**
  - gender - **String**

## โครงสร้างไฟล์

```
/lib
- main.dart
- models/body_profile.dart
- pages/bmr_page.dart
```

## รายละเอียดภายในไฟล์ body\_profile.dart

```
1 class BodyProfile {
2   · String gender;
3   · int age;
4   · double weight;
5   · double height;
6
7   · BodyProfile({
8     · · required this.gender,
9     · · required this.age,
10    · · required this.weight,
11    · · required this.height,
12    · · });
13 }
```

# ทดสอบความเข้าใจ (ต่อ)

- สร้างแบบฟอร์มสำหรับกรอกข้อมูล ในไฟล์ **bmr\_page.dart** ภายในโฟลเดอร์ **pages** เพื่อนำไปใช้ในการคำนวณ ค่า BMR โดยแบบฟอร์มประกอบด้วย น้ำหนัก ส่วนสูง อายุ เพศ (ชาย/หญิง) และมี 2 ปุ่มสำหรับกดเพื่อเริ่มคำนวณ และการ reset ค่า
- ให้นำข้อมูลที่กรอกเก็บลงในตัวแปรภายในคลาส **BodyProfile** ที่สร้างไว้
- ลักษณะหน้าจอที่ต้องการ ให้แสดงมีดังนี้

\* ให้ทำการ Validate ความถูกต้องของข้อมูลให้ครบถ้วน การ validate ให้ใช้ Library เสริมในการทำงาน

### BMR Calculator

Gender:

☒ Male ☐ Female

Weight (kg)

65

Height (cm)

170

Age:(years)

0 25 100

CalculateReset

เก็บลง gender ('male'/'female')

เก็บลง weight

เก็บลง height

เก็บลง age

\* ยังไม่ต้องทำการคำนวณ เพียงสร้างฟอร์มกรอกข้อมูลเท่านั้น (รอนำฟอร์มนี้ไปใช้งานในคาบเรียนถัดไป)

Q/A